6. Special Bits: setuid, setgid, sticky

06. Special Bits: setuid, setgid, sticky

Phase: Phase 2 — Permissions and identity
Estimated time: 1 day
Slug: `06-special-perms`

Learning objectives

- Explain setuid/setgid/sticky bit purposes
- Recognize them in `ls -l`
- Know security risks of setuid binaries
- Use sticky on shared directories like /tmp

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Plain read/write/execute bits are not enough for shared systems. Passwords must be changed by ordinary users even though `/etc/shadow` is root-only; `/tmp` must be writable by everyone without letting users delete each other's files; project directories should keep one group for all new files. Special bits solve these problems, and misusing them is also a classic privilege-escalation path.

Core concepts

There are three extra bits, shown as a fourth leading octal digit (`4755`, `2775`, `1777`).

Bit	Octal	On a file	On a directory	Shown as
setuid	4000	Runs with the file owner's UID	Ignored on Linux	`s` in user x slot
setgid	2000	Runs with the file's group	New entries inherit the directory's group	`s` in group x slot
sticky	1000	Ignored on modern Linux	Only owner of an entry (or dir owner or root) may delete or rename it	`t` in others x slot

A capital `S` or `T` means the special bit is set but the underlying execute bit is not.

```
/tmp   drwxrwxrwt  root root
           ^^^^^^^^^
      anyone may create files;
      sticky 't': only the file's owner
```

```
       (or root / dir owner) may remove them

  /usr/bin/passwd  -rwsr-xr-x root root
                      ^
       runs as root even when alice starts it,
       so it can update /etc/shadow
```

With setuid, the process has an *effective* UID of the file owner while the real UID remains the caller. The kernel ignores setuid on interpreted scripts (`#!` files) for safety, so it only works on compiled binaries.

Because setuid-root programs are high-value targets, keep their number small and audit them with `find`.

Worked examples

Spot the bits:

```
ls -ld /tmp
# sample output: drwxrwxrwt 14 root root 4096 Oct 21 10:00 /tmp
ls -l /usr/bin/passwd
# sample output: -rwsr-xr-x 1 root root 59976 Feb  6  2024 /usr/bin/passwd
stat -c '%a %n' /tmp /usr/bin/passwd
# sample output: 1777 /tmp
# sample output: 4755 /usr/bin/passwd
```

Sticky directory behavior:

```
mkdir /tmp/shared && chmod 1777 /tmp/shared
touch /tmp/shared/mine
sudo -u nobody rm /tmp/shared/mine
# sample output: rm: cannot remove '/tmp/shared/mine': Operation not permitted
```

Even though the directory is world-writable, `nobody` is not the owner, so deletion is refused.

Setgid directory for a team:

```
sudo mkdir /srv/team && sudo chgrp devs /srv/team
sudo chmod 2775 /srv/team
touch /srv/team/doc && ls -l /srv/team/doc
# sample output: -rw-r--r-- 1 student devs 0 Oct 21 10:05 /srv/team/doc
```

The new file got group `devs`, not the creator's primary group.

Audit setuid files:

```
find /usr/bin -perm -4000 -type f 2>/dev/null
# sample output: /usr/bin/passwd
# sample output: /usr/bin/sudo
# sample output: /usr/bin/su
```

Mini-lab

1. Run `ls -ld /tmp /usr/bin/passwd /usr/bin/sudo` and identify the `t` and `s` letters.

2. As root create `mkdir /tmp/lab6 && chmod 1777 /tmp/lab6`.
3. Create a file as your user, then try deleting it as another user: `sudo -u nobody rm /tmp/lab6/f`.
4. Remove the sticky bit with `chmod -t /tmp/lab6` and repeat; now deletion succeeds if the directory is writable.
5. Create `mkdir /tmp/g6 && chgrp users /tmp/g6 && chmod 2775 /tmp/g6`; add a file and check the group.
6. Set `chmod u+s somefile` on a harmless copy of `/bin/true` and view `ls -l`; notice the `s`. Then `chmod u-s` it.
7. Find setgid files: `find /usr -perm -2000 -type f 2>/dev/null | head`.
8. Convert `4755`, `2770`, `1777` to symbolic form by hand.

Decision guide / failure modes

Use sticky on shared writable directories (`/tmp`, upload spools); use setgid on directories for team collaboration; use setuid only for small audited binaries that truly need privilege, and prefer `sudo` or capabilities (`setcap`) for new designs.

Common failures:

- Seeing `S` or `T` and assuming it is fine: execute is missing, so the bit's effect may be broken.
- Setting setuid on a shell script and expecting it to work; Linux ignores it.
- Leaving a world-writable directory without sticky bit, so users delete each other's files.
- `chmod -R 2775` also setting setgid on files; apply to directories only.
- Backups or copies that drop special bits; check with `stat` after restoring.

Common mistakes

- Creating custom setuid root binaries casually
- World-writable setuid files (critical vulnerability)
- Ignoring sticky bit issues on shared drop folders

Exercises

Work in `exercises/06-special-perms/`. Read that folder's `README.md` fully before coding.

```
cd exercises/06-special-perms
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement describe_sticky(mode). The input is a string, either an octal mode with four digits like "1777" or a symbolic string like "drwxrwxrwt". Decide whether the sticky bit is set. If it is set, return the string "owner-delete-only", describing its effect on a directory: only the owner of an entry (or the directory owner or root) may delete or rename it. If the sticky bit is not set, return "normal". In octal the sticky bit is the 1 in the leading digit of four digits; in symbolic form it is a "t" or "T" as the last character of the mode.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/06-special-perms/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What does sticky do on a directory?
2. Why does `passwd` use setuid?
3. Name one risk of setuid

Further reading

- https://man7.org/linux/man-pages/man1/chmod.1.html